Ex6a: Exploratory Data Analysis – Netflix Titles

Dataset: Netflix Movies and TV Shows (Kaggle) – `netflix_titles.csv`

Upload `netflix_titles.csv` to the Colab **Files** panel before running. If the file is not uploaded, the notebook downloads it from Kaggle automatically.

```
!pip install -q wordcloud

import os
from collections import Counter
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud, STOPWORDS
import warnings
warnings.filterwarnings('ignore')

sns.set_style('whitegrid')
pd.set_option('display.max_columns', None)
```

Task 1: Load the Netflix titles dataset into a DataFrame

```
file_name = "netflix_titles.csv"
if not os.path.exists(file_name):  # file not uploaded -> download from Kaggle
    import kagglehub
    path = kagglehub.dataset_download("shivamb/netflix-shows")
    file_name = os.path.join(path, "netflix_titles.csv")

df = pd.read_csv(file_name)
print("Dataset loaded:", df.shape)
```

```
Dataset loaded: (8807, 12)
```

Task 2: Explore the Dataset

```
print("First five records:")
display(df.head())
print("Last five records:")
display(df.tail())
```

First five records:

	show_id	type	title	director	cast	country	date_added	release_year	rating
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021	TV-MA
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2021	TV-MA
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021	TV-MA
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2021	TV-MA

	duration	listed_in	description
0	90 min	Documentaries	As her father nears the end of his life, filmm...
1	2 Seasons	International TV Shows, TV Dramas, TV Mysteries	After crossing paths at a party, a Cape Town t...
2	1 Season	Crime TV Shows, International TV Shows, TV Act...	To protect his family from a powerful drug lor...
3	1 Season	Docuseries, Reality TV	Feuds, flirtations and toilet talk go down amo...
4	2 Seasons	International TV Shows, Romantic TV Shows, TV ...	In a city of coaching centers known to train I...

Last five records:

	show_id	type	title	director	cast	country	date_added	release_year	rating
8802	s8803	Movie	Zodiac	David Fincher	Mark Ruffalo, Jake Gyllenhaal, Robert Downey J...	United States	November 20, 2019	2007	R
8803	s8804	TV Show	Zombie Dumb	NaN	NaN	NaN	July 1, 2019	2018	TV-Y7
8804	s8805	Movie	Zombieland	Ruben Fleischer	Jesse Eisenberg, Woody Harrelson, Emma Stone, ...	United States	November 1, 2019	2009	R

	show_id	type	title	director	cast	country	date_added	release_year	rating
8805	s8806	Movie	Zoom	Peter Hewitt	Tim Allen, Courteney Cox, Chevy Chase, Kate Ma...	United States	January 11, 2020	2006	PG
8806	s8807	Movie	Zubaan	Mozez Singh	Vicky Kaushal, Sarah-Jane Dias, Raaghav Chanan...	India	March 2, 2019	2015	TV-14

	duration	listed_in	description
8802	158 min	Cult Movies, Dramas, Thrillers	A political cartoonist, a crime reporter and a...
8803	2 Seasons	Kids' TV, Korean TV Shows, TV Comedies	While living alone in a spooky town, a young g...
8804	88 min	Comedies, Horror Movies	Looking to survive in a world taken over by zo...
8805	88 min	Children & Family Movies, Comedies	Dragged from civilian life, a former superhero...
8806	111 min	Dramas, International Movies, Music & Musicals	A scrappy but poor boy worms his way into a ty...

```
print("Shape:", df.shape)
print("\nColumn names:", df.columns.tolist())
print("\nDataset information:")
df.info()
```

```
Shape: (8807, 12)

Column names: ['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added', 'release_year', 'rating', 'duration',
'listed_in', 'description']

Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   show_id       8807 non-null   str
 1   type          8807 non-null   str
 2   title         8807 non-null   str
 3   director      6173 non-null   str
 4   cast          7982 non-null   str
 5   country       7976 non-null   str
 6   date_added    8797 non-null   str
 7   release_year  8807 non-null   int64
 8   rating        8803 non-null   str
 9   duration      8804 non-null   str
 10  listed_in     8807 non-null   str
 11  description   8807 non-null   str
dtypes: int64(1), str(11)
memory usage: 825.8 KB
```

```
print("Summary statistics:")
display(df.describe(include='all').T)
```

Summary statistics:

	count	unique	top	freq	mean	std	min	25%	50%
show_id	8807	8807	s1	1	NaN	NaN	NaN	NaN	NaN
type	8807	2	Movie	6131	NaN	NaN	NaN	NaN	NaN
title	8807	8807	Dick Johnson Is Dead	1	NaN	NaN	NaN	NaN	NaN
director	6173	4528	Rajiv Chilaka	19	NaN	NaN	NaN	NaN	NaN
cast	7982	7692	David Attenborough	19	NaN	NaN	NaN	NaN	NaN
country	7976	748	United States	2818	NaN	NaN	NaN	NaN	NaN
date_added	8797	1767	January 1, 2020	109	NaN	NaN	NaN	NaN	NaN
release_year	8807.0	NaN	NaN	NaN	2014.180198	8.819312	1925.0	2013.0	2017.0
rating	8803	17	TV-MA	3207	NaN	NaN	NaN	NaN	NaN
duration	8804	220	1 Season	1793	NaN	NaN	NaN	NaN	NaN
listed_in	8807	514	Dramas, International Movies	362	NaN	NaN	NaN	NaN	NaN
description	8807	8775	Paranormal activity at a lush, abandoned prope...	4	NaN	NaN	NaN	NaN	NaN

	75%	max
show_id	NaN	NaN
type	NaN	NaN
title	NaN	NaN
director	NaN	NaN
cast	NaN	NaN
country	NaN	NaN
date_added	NaN	NaN
release_year	2019.0	2021.0
rating	NaN	NaN
duration	NaN	NaN

	75%	max
listed_in	NaN	NaN
description	NaN	NaN

```
print("Missing values in each column:")
missing = pd.DataFrame({'Missing Count': df.isnull().sum(),
                        'Missing %': (df.isnull().mean() * 100).round(2)})
display(missing)
print("Number of duplicate records:", df.duplicated().sum())
print("\nUnique values in each column:")
print(df.nunique())
```

```
Missing values in each column:
```

	Missing Count	Missing %
show_id	0	0.00
type	0	0.00
title	0	0.00
director	2634	29.91
cast	825	9.37
country	831	9.44
date_added	10	0.11
release_year	0	0.00
rating	4	0.05
duration	3	0.03
listed_in	0	0.00
description	0	0.00

```
Number of duplicate records: 0

Unique values in each column:
show_id         8807
type               2
title           8807
director        4528
cast            7692
country          748
date_added      1767
release_year      74
rating            17
duration         220
listed_in        514
description     8775
dtype: int64
```

Data preparation for the analysis - 3 rows have the movie duration (e.g. "74 min") wrongly stored in the `rating` column → move it to `duration`. - Convert `date_added` to datetime and extract the year and month added. - Extract the numeric part of `duration` (minutes for movies, seasons for TV shows).

```
wrong_rating = df['rating'].astype(str).str.contains('min')
print("Rows with duration stored in rating:", wrong_rating.sum())
df.loc[wrong_rating, 'duration'] = df.loc[wrong_rating, 'rating']
df.loc[wrong_rating, 'rating'] = np.nan

df['date_added'] = pd.to_datetime(df['date_added'].str.strip(), format='%B %d, %Y', errors='coerce')
df['year_added'] = df['date_added'].dt.year
df['month_added'] = df['date_added'].dt.month_name()

df['duration_num'] = df['duration'].str.extract(r'(\d+)', expand=False).astype(float)

display(df[['title', 'type', 'date_added', 'year_added', 'month_added', 'duration', 'duration_num']].head())
```

```
Rows with duration stored in rating: 3
```

	title	type	date_added	year_added	month_added	duration	duration_num
0	Dick Johnson Is Dead	Movie	2021-09-25	2021.0	September	90 min	90.0
1	Blood & Water	TV Show	2021-09-24	2021.0	September	2 Seasons	2.0
2	Ganglands	TV Show	2021-09-24	2021.0	September	1 Season	1.0
3	Jailbirds New Orleans	TV Show	2021-09-24	2021.0	September	1 Season	1.0
4	Kota Factory	TV Show	2021-09-24	2021.0	September	2 Seasons	2.0

```
def split_explode(column, data=df):
    """Split comma-separated values (e.g. 'United States, India') into one row per value."""
    return (data[column].dropna().str.split(',').explode().str.strip()
            .replace('', np.nan).dropna())
```

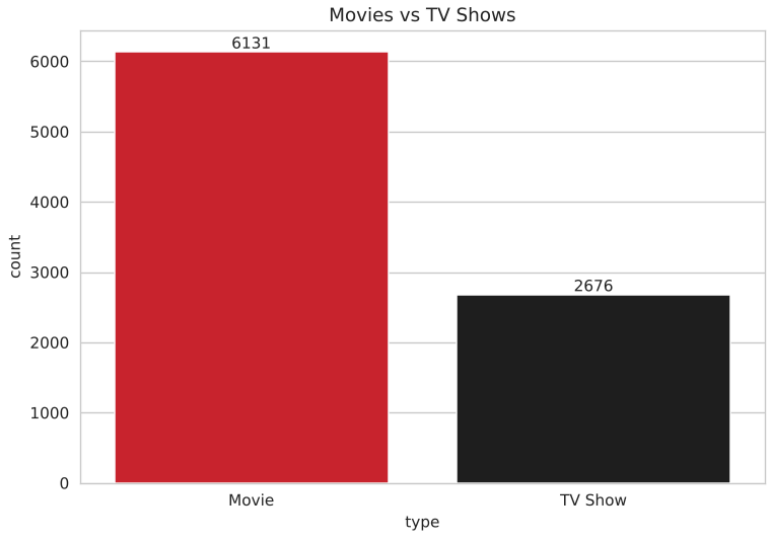
Task 3: Content Distribution Analysis

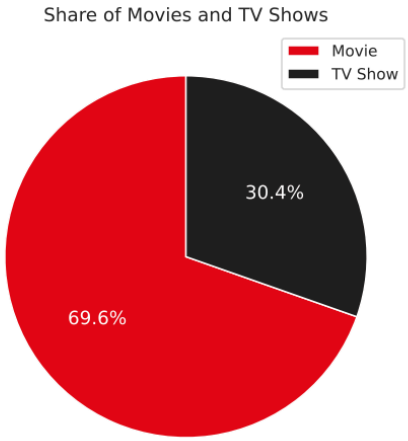
3a. How many Movies vs TV Shows are in the dataset?

```
type_counts = df['type'].value_counts()
display(type_counts.to_frame('Count'))

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
sns.countplot(data=df, x='type', hue='type', palette=['#E50914', '#221F1F'], legend=False, ax=axes[0])
for container in axes[0].containers:
    axes[0].bar_label(container)
axes[0].set_title('Movies vs TV Shows')
axes[1].pie(type_counts, labels=type_counts.index, autopct='%1.1f%%', colors=['#E50914', '#221F1F'],
            startangle=90, textprops={'color': 'white', 'fontsize': 12})
axes[1].legend(type_counts.index, loc='upper right')
axes[1].set_title('Share of Movies and TV Shows')
plt.tight_layout()
plt.show()
```

type	Count
Movie	6131
TV Show	2676





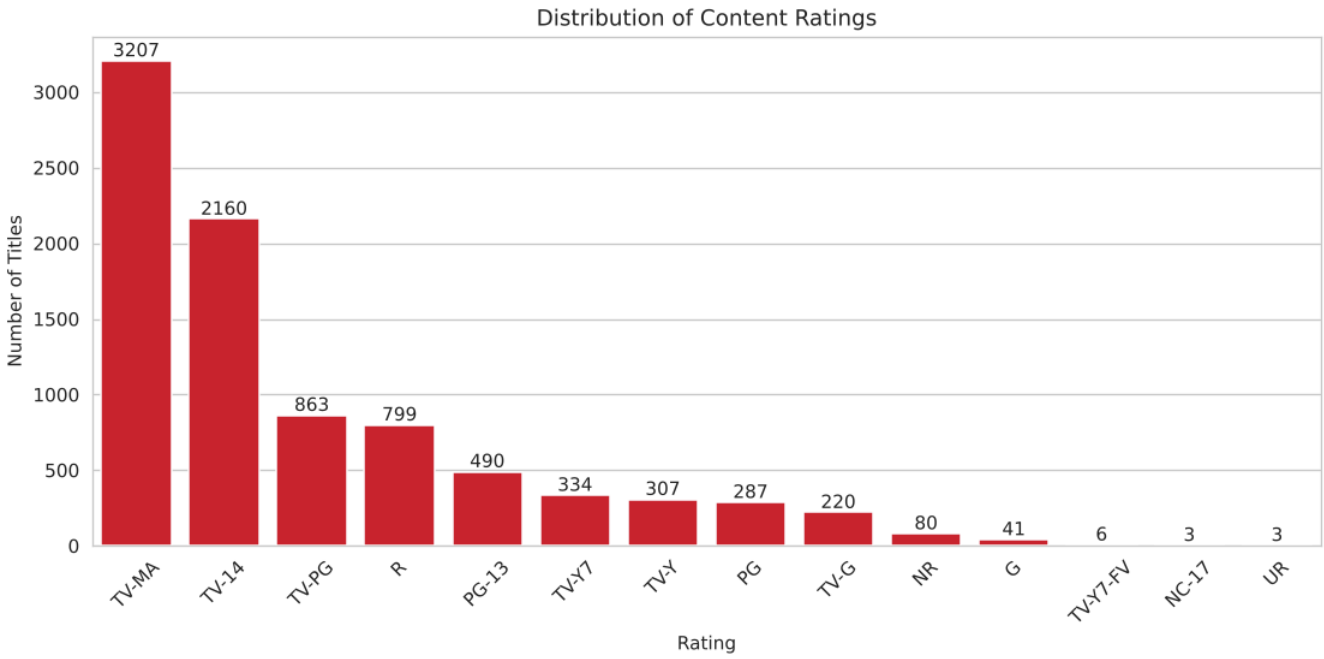
3b. What is the distribution of ratings?

```
rating_counts = df['rating'].value_counts()
display(rating_counts.to_frame('Count'))

plt.figure(figsize=(12, 5))
ax = sns.countplot(data=df, x='rating', order=rating_counts.index, color='#E50914')
ax.bar_label(ax.containers[0])
plt.title('Distribution of Content Ratings')
plt.xlabel('Rating')
plt.ylabel('Number of Titles')
plt.xticks(rotation=45)
plt.show()
```

rating	Count
TV-MA	3207
TV-14	2160
TV-PG	863
R	799
PG-13	490
TV-Y7	334
TV-Y	307
PG	287
TV-G	220
NR	80
G	41
TV-Y7-FV	6

rating	Count
NC-17	3
UR	3

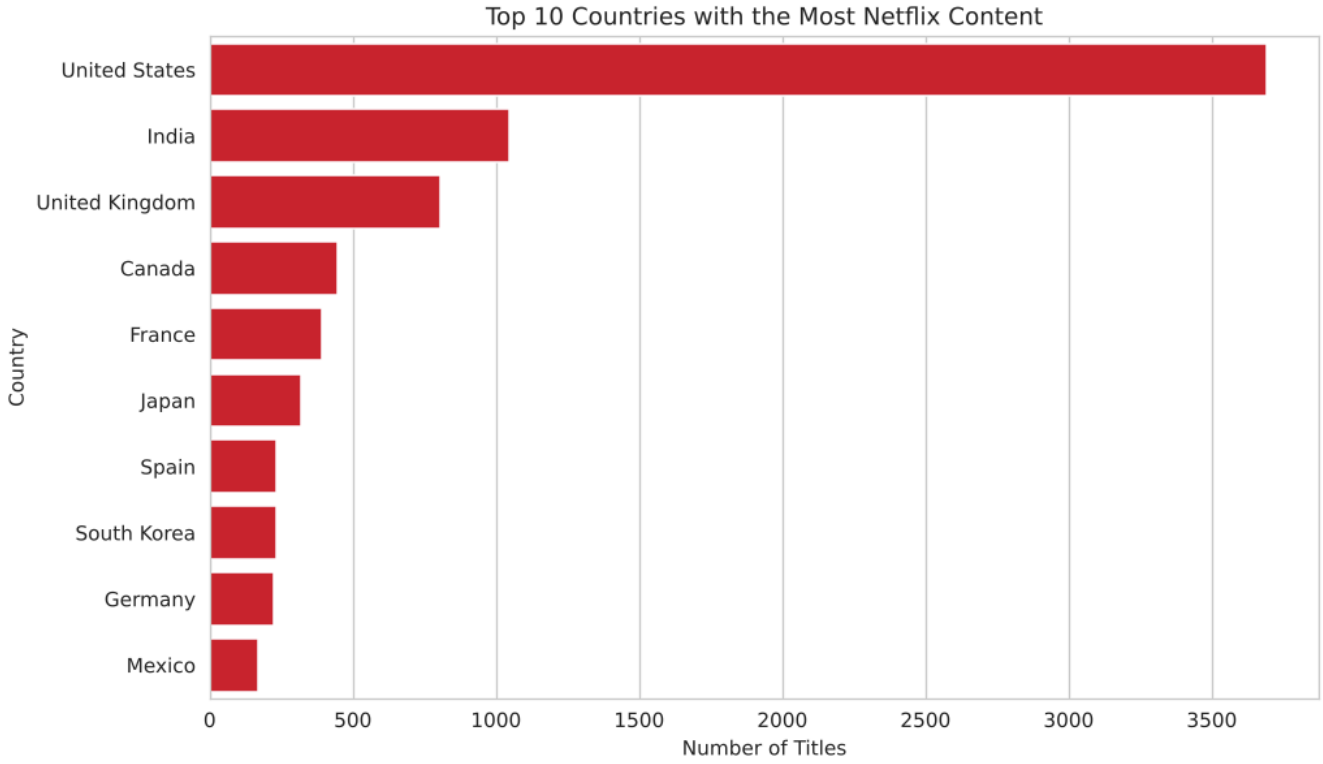


3c. Which countries have the most Netflix content?

```
country_counts = split_explode('country').value_counts().head(10)
display(country_counts.to_frame('Number of Titles'))

plt.figure(figsize=(10, 6))
sns.barplot(x=country_counts.values, y=country_counts.index, color='#E50914')
plt.title('Top 10 Countries with the Most Netflix Content')
plt.xlabel('Number of Titles')
plt.ylabel('Country')
plt.show()
```

country	Number of Titles
United States	3690
India	1046
United Kingdom	806
Canada	445
France	393
Japan	318
Spain	232
South Korea	231
Germany	226
Mexico	169



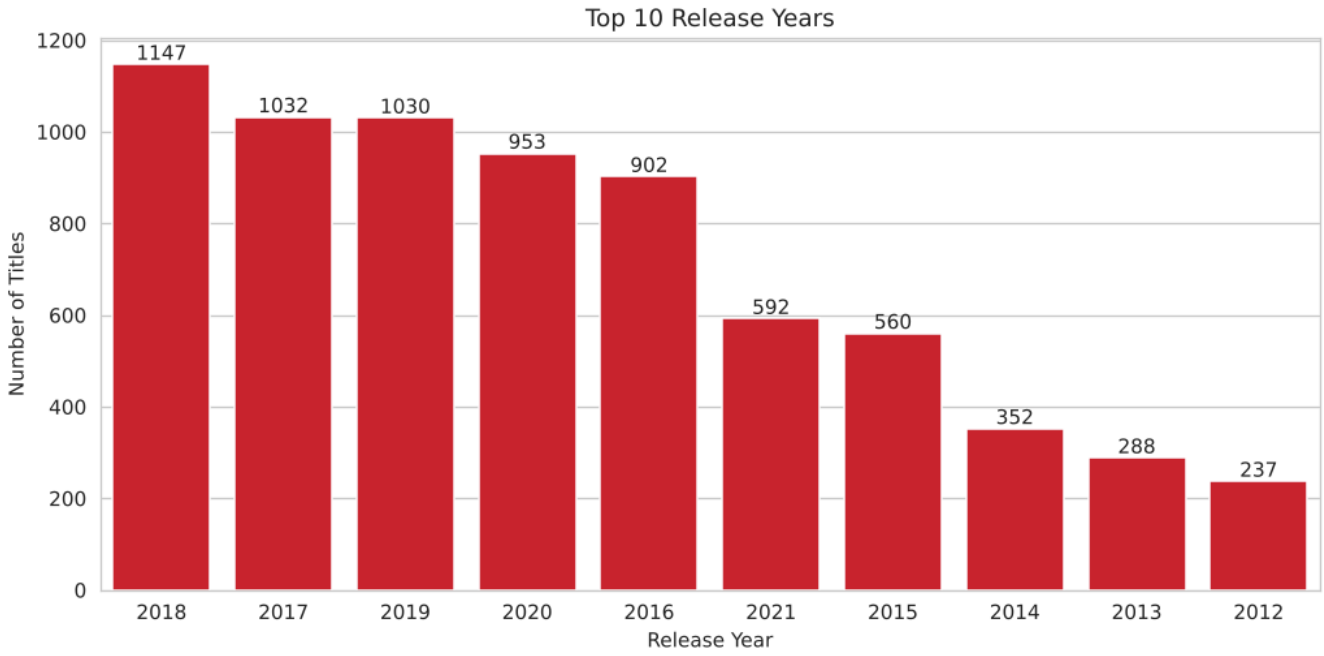
Task 4: Time-Based Trend Analysis

4a. In which years were the most shows/movies released?

```
release_counts = df['release_year'].value_counts().head(10)
display(release_counts.to_frame('Number of Titles'))

plt.figure(figsize=(11, 5))
ax = sns.barplot(x=release_counts.index.astype(str), y=release_counts.values,
                 order=release_counts.index.astype(str), color='#E50914')
ax.bar_label(ax.containers[0])
plt.title('Top 10 Release Years')
plt.xlabel('Release Year')
plt.ylabel('Number of Titles')
plt.show()
```

release_year	Number of Titles
2018	1147
2017	1032
2019	1030
2020	953
2016	902
2021	592
2015	560
2014	352
2013	288
2012	237

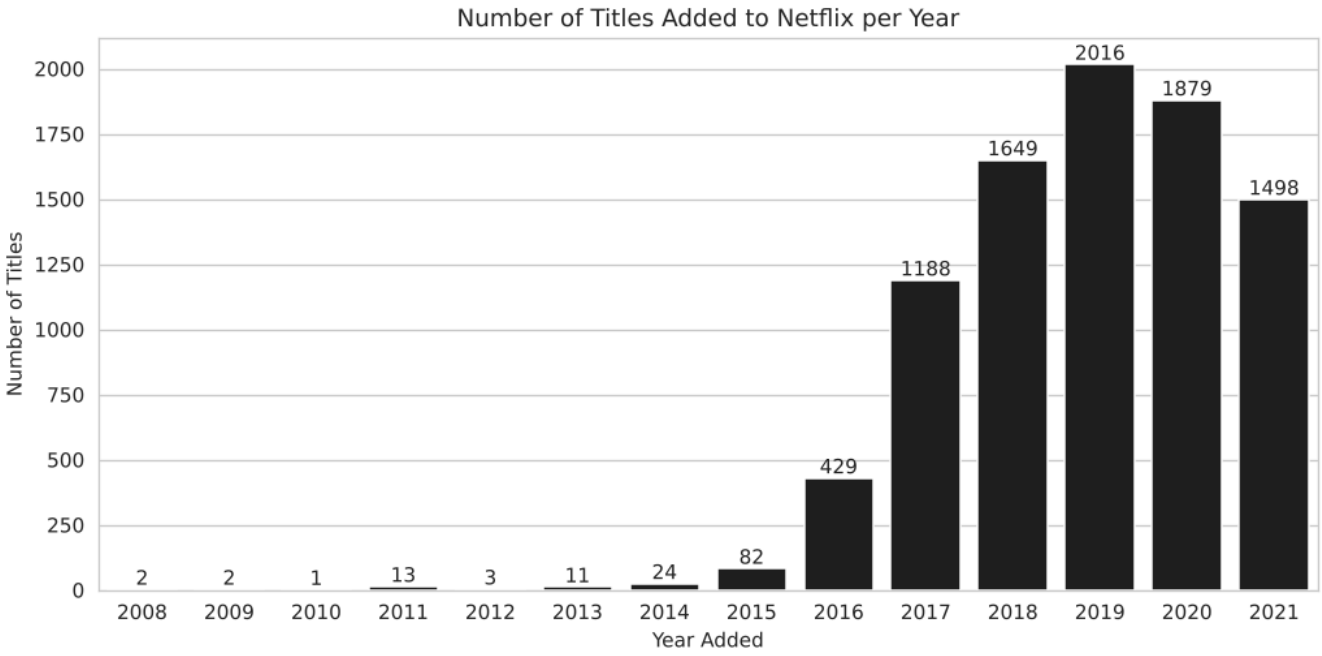


4b. In which years were titles added to Netflix?

```
added_counts = df['year_added'].value_counts().sort_index()
display(added_counts.astype(int).to_frame('Titles Added'))

plt.figure(figsize=(11, 5))
ax = sns.barplot(x=added_counts.index.astype(int).astype(str), y=added_counts.values, color='#221F1F')
ax.bar_label(ax.containers[0])
plt.title('Number of Titles Added to Netflix per Year')
plt.xlabel('Year Added')
plt.ylabel('Number of Titles')
plt.show()
```

year_added	Titles Added
2008.0	2
2009.0	2
2010.0	1
2011.0	13
2012.0	3
2013.0	11
2014.0	24
2015.0	82
2016.0	429
2017.0	1188
2018.0	1649
2019.0	2016
2020.0	1879
2021.0	1498



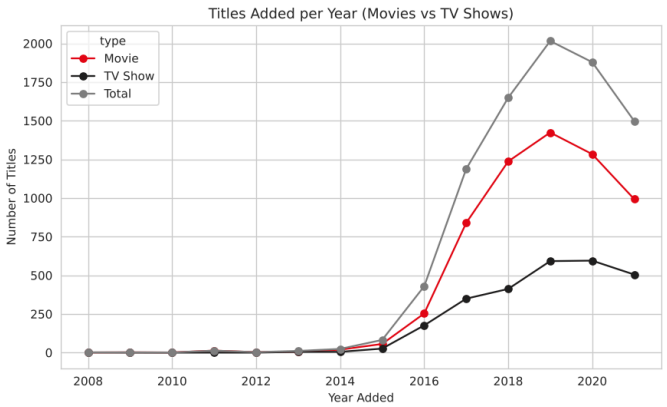
4c. How has Netflix's content growth trend changed over time?

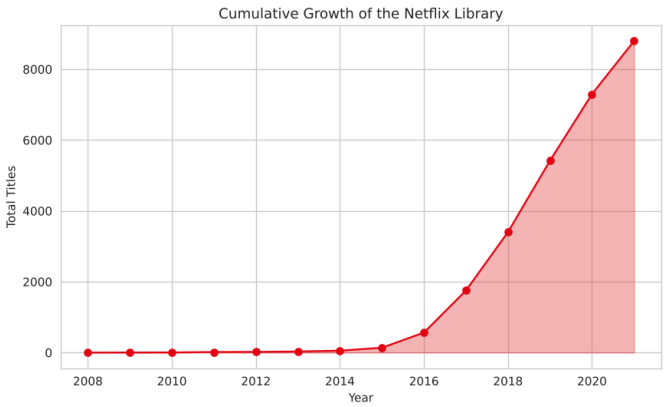
```
growth = df.groupby(['year_added', 'type']).size().unstack(fill_value=0)
growth.index = growth.index.astype(int)
growth['Total'] = growth.sum(axis=1)
growth['Cumulative Total'] = growth['Total'].cumsum()
display(growth)

fig, axes = plt.subplots(1, 2, figsize=(16, 5))
growth[['Movie', 'TV Show', 'Total']].plot(marker='o', ax=axes[0], color=['#E50914', '#221F1F', 'grey'])
axes[0].set_title('Titles Added per Year (Movies vs TV Shows)')
axes[0].set_xlabel('Year Added')
axes[0].set_ylabel('Number of Titles')

axes[1].fill_between(growth.index, growth['Cumulative Total'], color='#E50914', alpha=0.3)
axes[1].plot(growth.index, growth['Cumulative Total'], marker='o', color='#E50914')
axes[1].set_title('Cumulative Growth of the Netflix Library')
axes[1].set_xlabel('Year')
axes[1].set_ylabel('Total Titles')
plt.tight_layout()
plt.show()
```

type	Movie	TV Show	Total	Cumulative Total
year_added				
2008	1	1	2	2
2009	2	0	2	4
2010	1	0	1	5
2011	13	0	13	18
2012	3	0	3	21
2013	6	5	11	32
2014	19	5	24	56
2015	56	26	82	138
2016	253	176	429	567
2017	839	349	1188	1755
2018	1237	412	1649	3404
2019	1424	592	2016	5420
2020	1284	595	1879	7299
2021	993	505	1498	8797





Task 5: Duration & Content Type Analysis

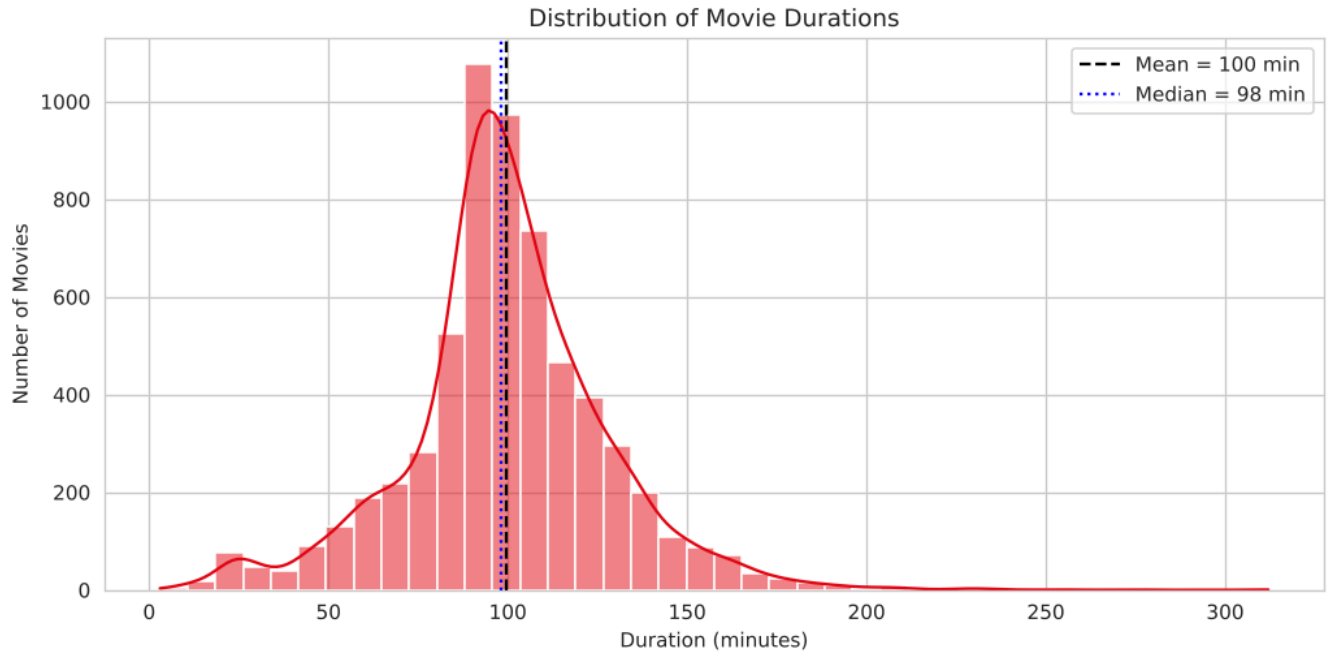
5a. What is the distribution of movie durations (in minutes)?

```
movies = df[df['type'] == 'Movie']
print("Movie duration statistics (minutes):")
display(movies['duration_num'].describe().round(2).to_frame('Duration (min)'))

plt.figure(figsize=(11, 5))
sns.histplot(movies['duration_num'], bins=40, kde=True, color='#E50914')
plt.axvline(movies['duration_num'].mean(), color='black', linestyle='--',
            label=f"Mean = {movies['duration_num'].mean():.0f} min")
plt.axvline(movies['duration_num'].median(), color='blue', linestyle=':',
            label=f"Median = {movies['duration_num'].median():.0f} min")
plt.title('Distribution of Movie Durations')
plt.xlabel('Duration (minutes)')
plt.ylabel('Number of Movies')
plt.legend()
plt.show()
```

Movie duration statistics (minutes):

	Duration (min)
count	6131.00
mean	99.56
std	28.29
min	3.00
25%	87.00
50%	98.00
75%	114.00
max	312.00

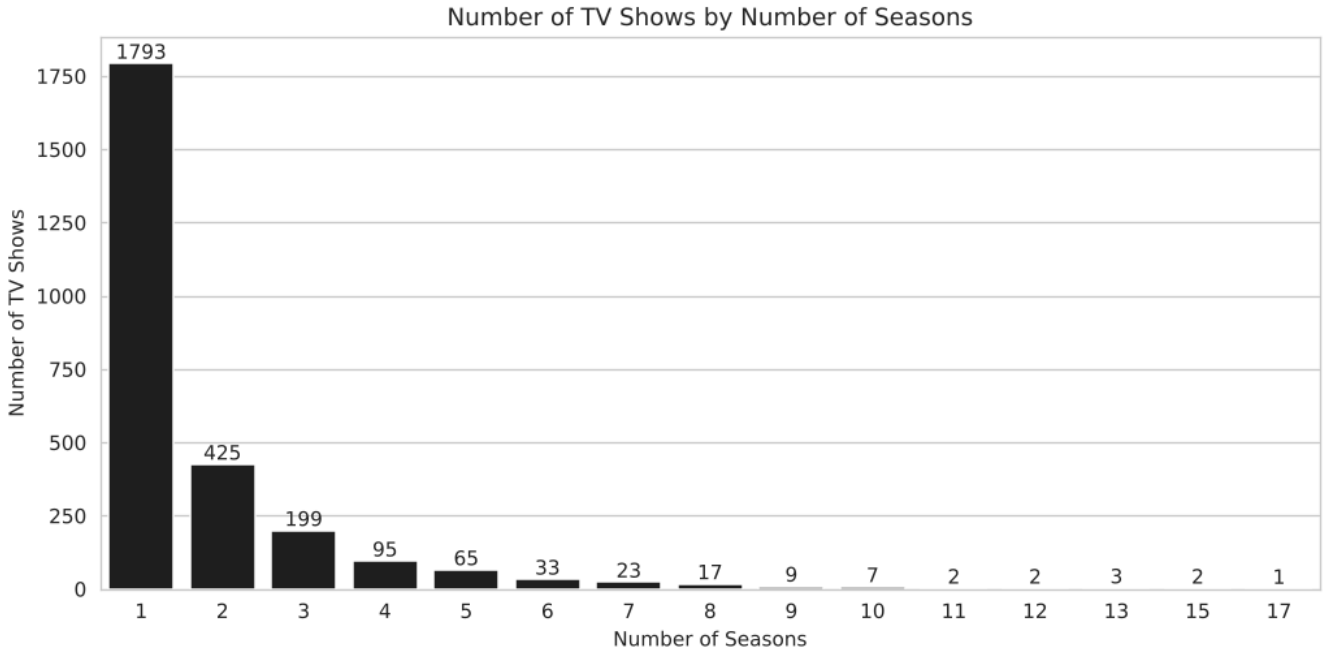


5b. How many TV shows exist per number of seasons?

```
tv_shows = df[df['type'] == 'TV Show']
season_counts = tv_shows['duration_num'].value_counts().sort_index()
season_counts.index = season_counts.index.astype(int)
display(season_counts.to_frame('Number of TV Shows'))

plt.figure(figsize=(11, 5))
ax = sns.barplot(x=season_counts.index.astype(str), y=season_counts.values, color='#221F1F')
ax.bar_label(ax.containers[0])
plt.title('Number of TV Shows by Number of Seasons')
plt.xlabel('Number of Seasons')
plt.ylabel('Number of TV Shows')
plt.show()
```

duration_num	Number of TV Shows
1	1793
2	425
3	199
4	95
5	65
6	33
7	23
8	17
9	9
10	7
11	2
12	2
13	3
15	2
17	1



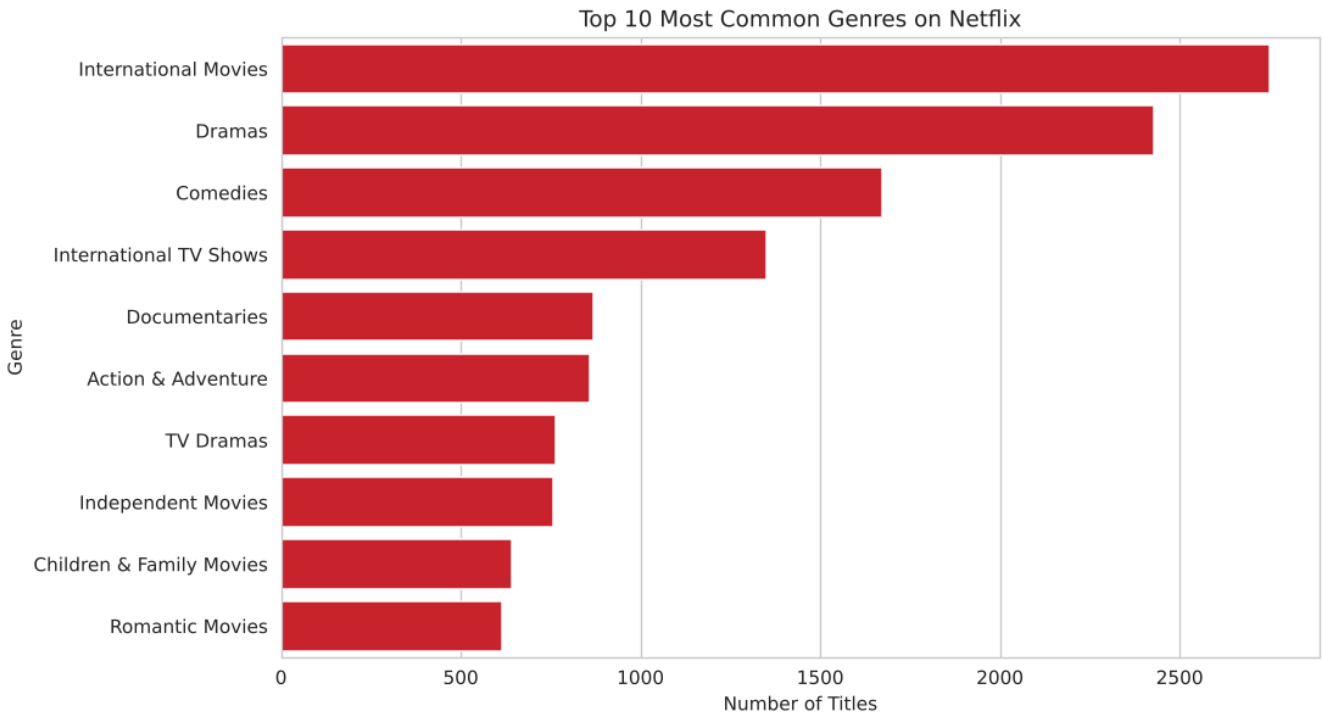
Task 6: Genre & Categories Analysis

6a. Top 10 most common genres

```
genre_counts = split_explode('listed_in').value_counts().head(10)
display(genre_counts.to_frame('Number of Titles'))

plt.figure(figsize=(10, 6))
sns.barplot(x=genre_counts.values, y=genre_counts.index, color='#E50914')
plt.title('Top 10 Most Common Genres on Netflix')
plt.xlabel('Number of Titles')
plt.ylabel('Genre')
plt.show()
```

listed_in	Number of Titles
International Movies	2752
Dramas	2427
Comedies	1674
International TV Shows	1351
Documentaries	869
Action & Adventure	859
TV Dramas	763
Independent Movies	756
Children & Family Movies	641
Romantic Movies	616

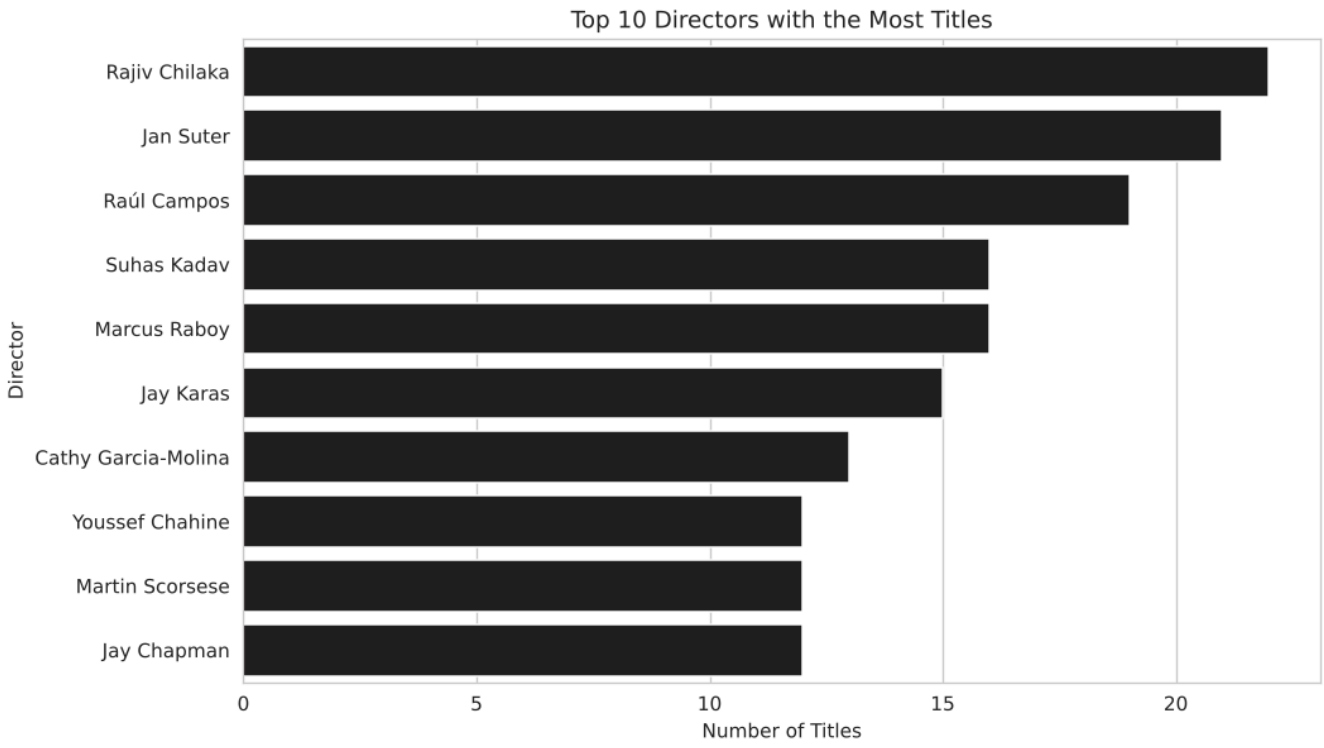


6b. Which directors have the most titles?

```
director_counts = split_explode('director').value_counts().head(10)
display(director_counts.to_frame('Number of Titles'))

plt.figure(figsize=(10, 6))
sns.barplot(x=director_counts.values, y=director_counts.index, color='#221F1F')
plt.title('Top 10 Directors with the Most Titles')
plt.xlabel('Number of Titles')
plt.ylabel('Director')
plt.show()
```

director	Number of Titles
Rajiv Chilaka	22
Jan Suter	21
Raúl Campos	19
Suhas Kadav	16
Marcus Raboy	16
Jay Karas	15
Cathy Garcia-Molina	13
Youssef Chahine	12
Martin Scorsese	12
Jay Chapman	12

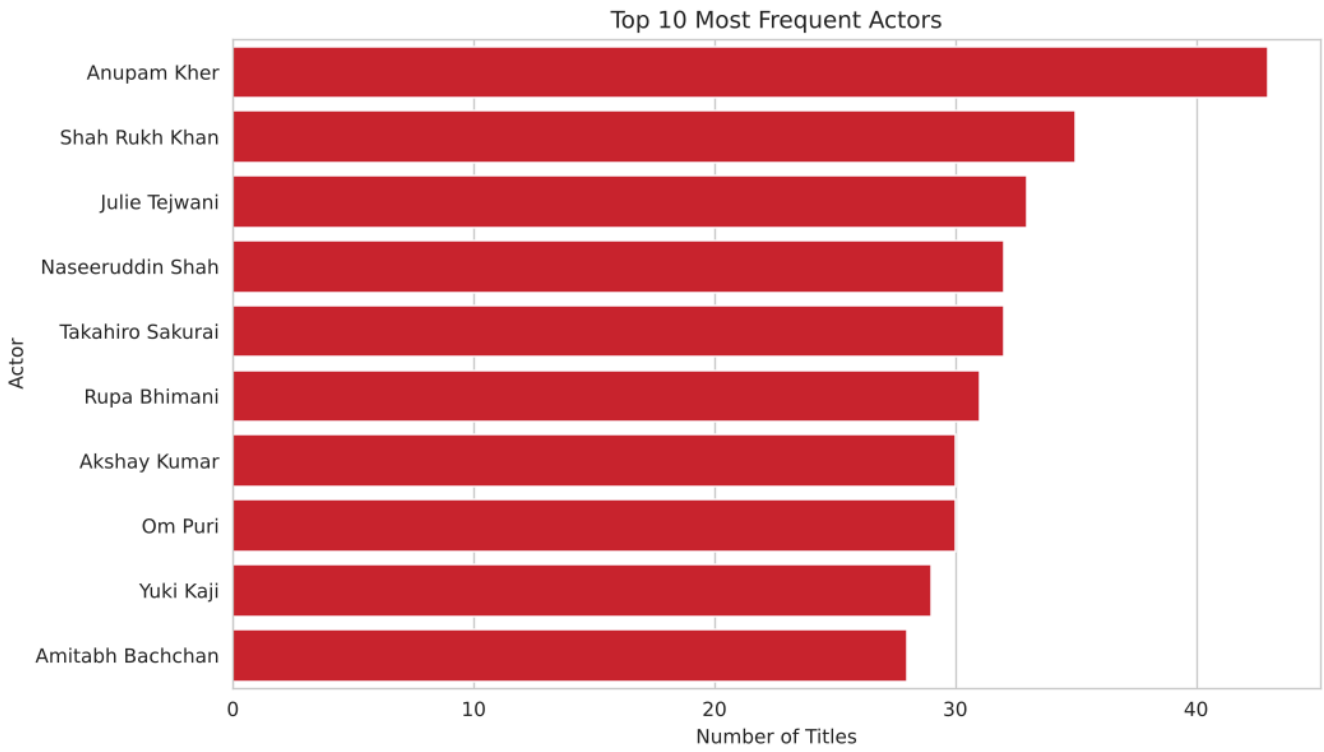


6c. Which actors appear most frequently?

```
actor_counts = split_explode('cast').value_counts().head(10)
display(actor_counts.to_frame('Number of Titles'))

plt.figure(figsize=(10, 6))
sns.barplot(x=actor_counts.values, y=actor_counts.index, color='#E50914')
plt.title('Top 10 Most Frequent Actors')
plt.xlabel('Number of Titles')
plt.ylabel('Actor')
plt.show()
```

cast	Number of Titles
Anupam Kher	43
Shah Rukh Khan	35
Julie Tejwani	33
Naseeruddin Shah	32
Takahiro Sakurai	32
Rupa Bhimani	31
Akshay Kumar	30
Om Puri	30
Yuki Kaji	29
Amitabh Bachchan	28



Task 7: Text & Descriptions

7a. Most common words in the description field (Word Cloud)

```
text = ' '.join(df['description'].dropna()).lower()
stopwords = set(STOPWORDS)

words = [w.strip('.,!?;:"\'()-') for w in text.split()]
words = [w for w in words if w and w.isalpha() and w not in stopwords and len(w) > 2]
top_words = pd.DataFrame(Counter(words).most_common(20), columns=['Word', 'Frequency'])
print("Top 20 most common words in descriptions:")
display(top_words)

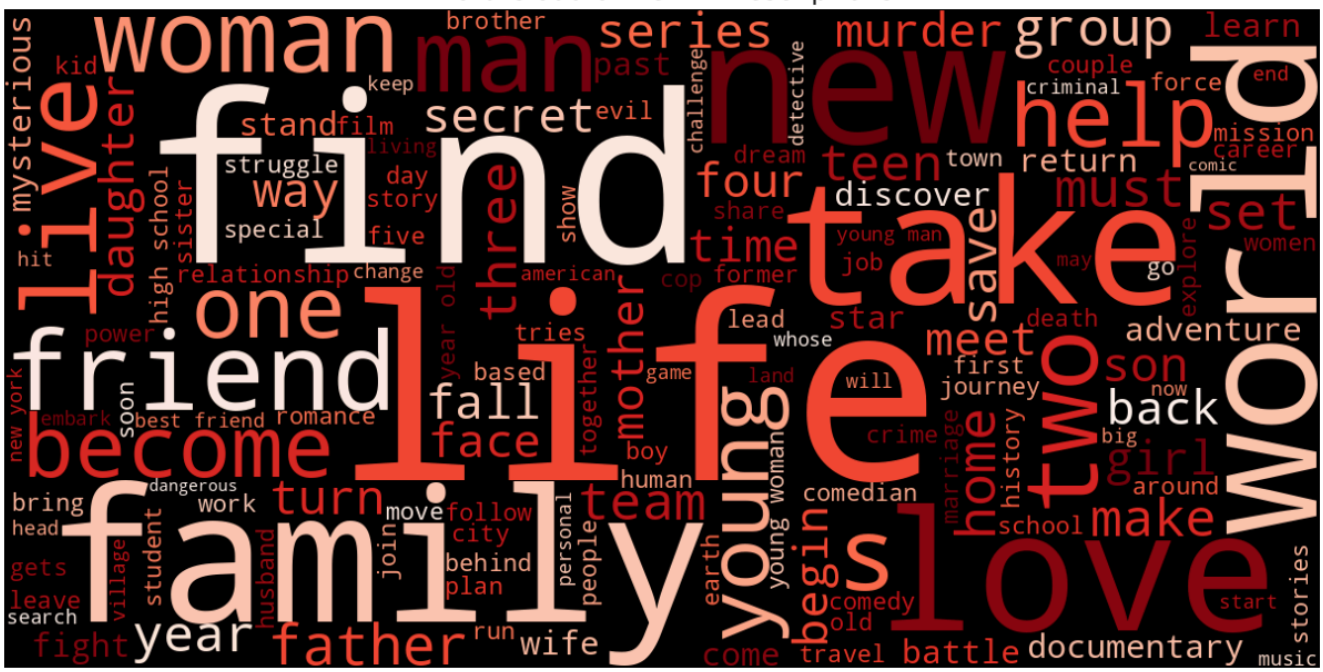
wordcloud = WordCloud(width=1200, height=600, background_color='black', colormap='Reds',
                      stopwords=stopwords, max_words=150).generate(text)
plt.figure(figsize=(15, 7))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud of Netflix Descriptions', fontsize=16)
plt.show()
```

```
Top 20 most common words in descriptions:
```

	Word	Frequency
0	life	774
1	young	728
2	new	699
3	family	570
4	love	497
5	two	495
6	man	491
7	world	491
8	woman	452
9	friends	442
10	must	398
11	series	394
12	documentary	361
13	one	346
14	find	341
15	finds	313
16	school	312
17	home	306
18	three	292

	Word	Frequency
19	help	291

Word Cloud of Netflix Descriptions



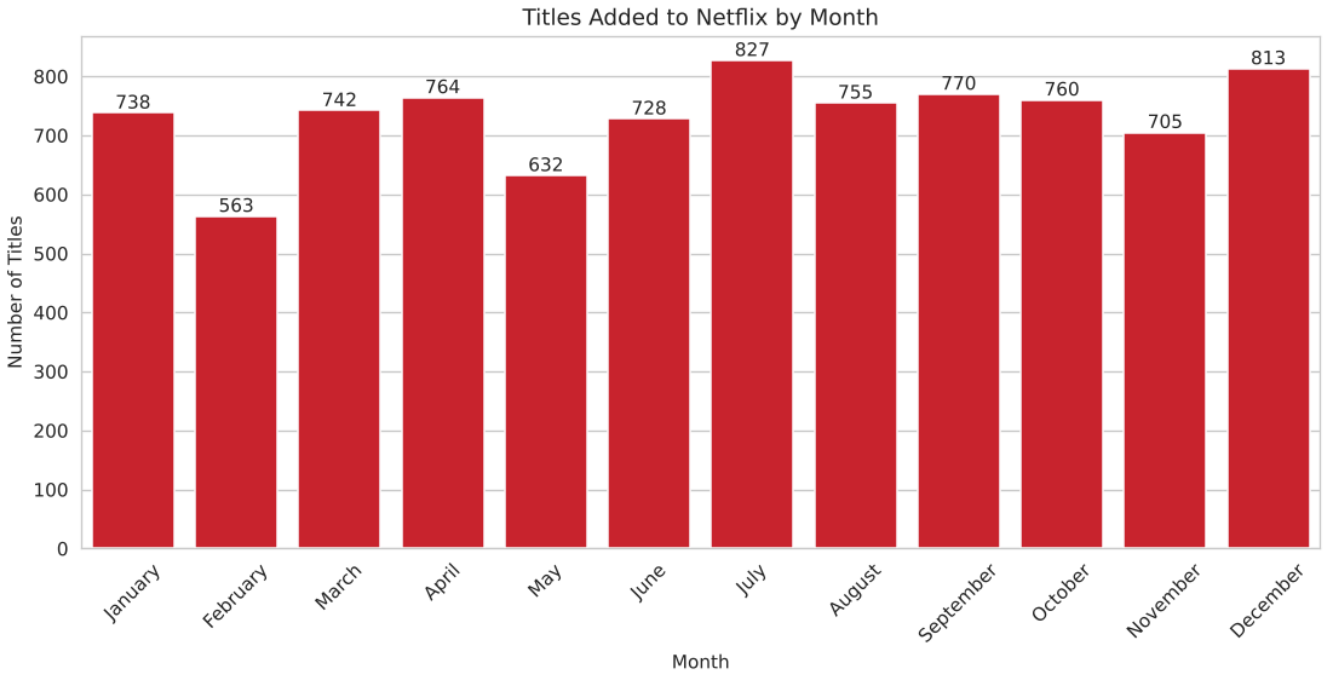
Task 8: In which months of the year does Netflix add the most content?

```
month_order = ['January', 'February', 'March', 'April', 'May', 'June', 'July',
               'August', 'September', 'October', 'November', 'December']
month_counts = df['month_added'].value_counts().reindex(month_order)
display(month_counts.astype(int).to_frame('Titles Added'))
print("Month with the most content added:", month_counts.idxmax())

plt.figure(figsize=(12, 5))
ax = sns.barplot(x=month_counts.index, y=month_counts.values, color='#E50914')
ax.bar_label(ax.containers[0])
plt.title('Titles Added to Netflix by Month')
plt.xlabel('Month')
plt.ylabel('Number of Titles')
plt.xticks(rotation=45)
plt.show()
```

	Titles Added
month_added	
January	738
February	563
March	742
April	764
May	632
June	728
July	827
August	755
September	770
October	760
November	705
December	813

```
Month with the most content added: July
```



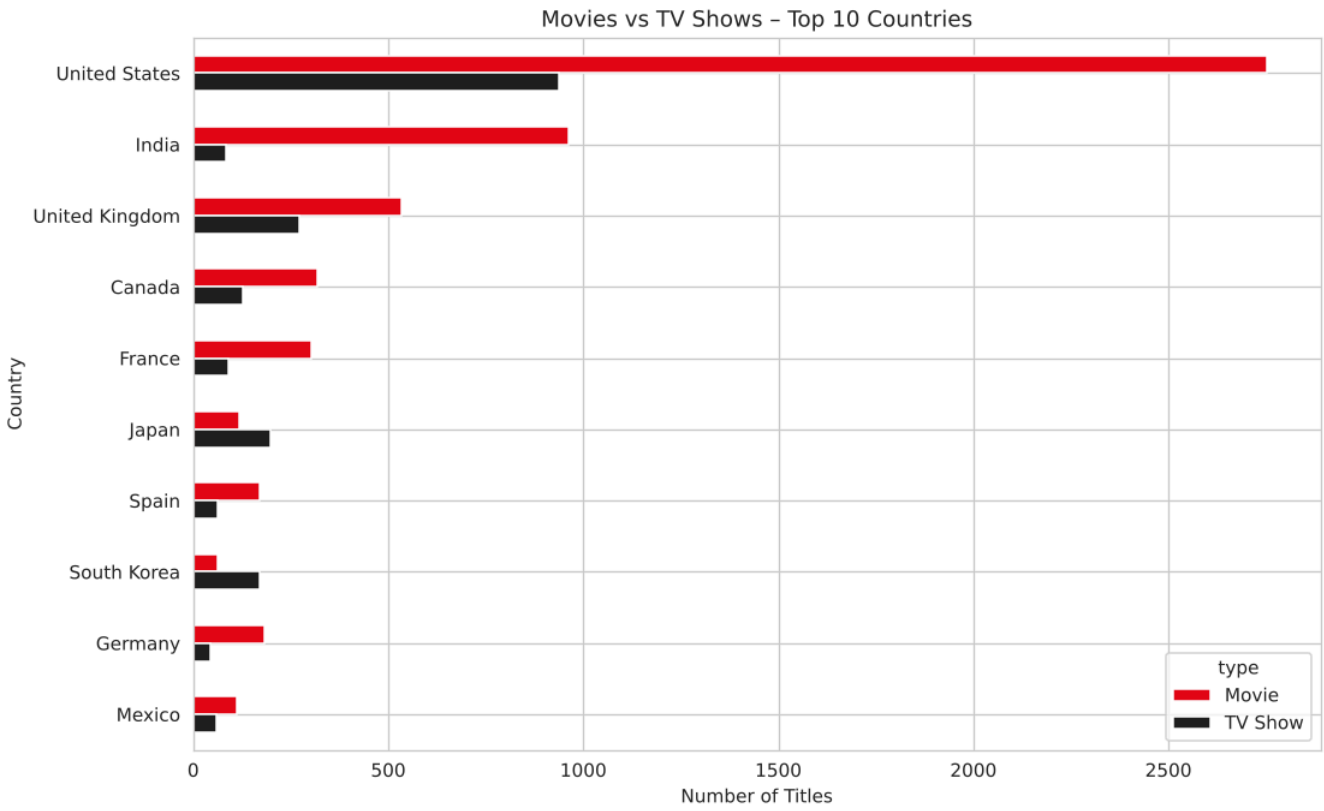
Task 9: Which countries produce more movies vs TV shows?

```
country_type = df[['country', 'type']].dropna().copy()
country_type['country'] = country_type['country'].str.split(',')
country_type = country_type.explode('country').reset_index(drop=True)
country_type['country'] = country_type['country'].str.strip()
country_type = country_type[country_type['country'] != '']

top_countries = country_type['country'].value_counts().head(10).index
country_crosstab = (pd.crosstab(country_type['country'], country_type['type'])
                    .loc[top_countries])
country_crosstab['Produces More'] = np.where(country_crosstab['Movie'] >= country_crosstab['TV Show'],
                                             'Movies', 'TV Shows')
display(country_crosstab)

country_crosstab[['Movie', 'TV Show']].plot(kind='barh', figsize=(11, 7), color=['#E50914', '#221F1F'])
plt.gca().invert_yaxis()
plt.title('Movies vs TV Shows — Top 10 Countries')
plt.xlabel('Number of Titles')
plt.ylabel('Country')
plt.show()
```

type	Movie	TV Show	Produces More
country			
United States	2752	938	Movies
India	962	84	Movies
United Kingdom	534	272	Movies
Canada	319	126	Movies
France	303	90	Movies
Japan	119	199	TV Shows
Spain	171	61	Movies
South Korea	61	170	TV Shows
Germany	182	44	Movies
Mexico	111	58	Movies



Task 10: What are the most common ratings for Movies vs TV Shows?

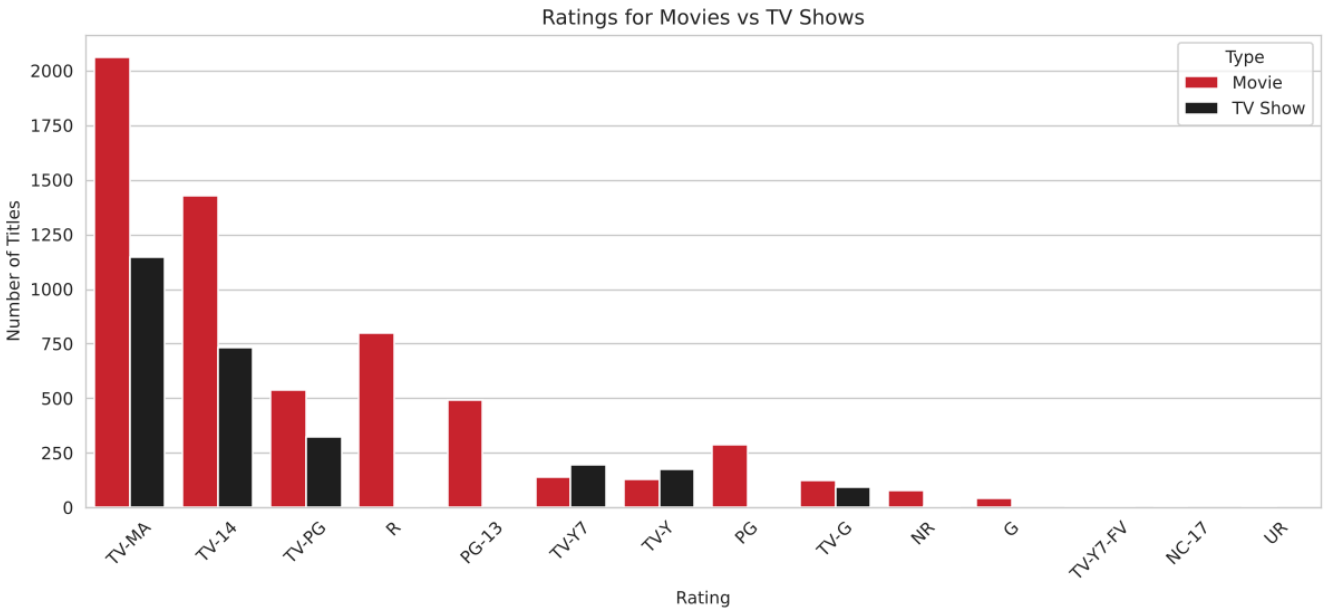
```
rating_type = pd.crosstab(df['rating'], df['type'])
rating_type = rating_type.loc[rating_type.sum(axis=1).sort_values(ascending=False).index]
display(rating_type)

print("Top 5 ratings for Movies  :", df[df['type'] == 'Movie']['rating'].value_counts().head(5).to_dict())
print("Top 5 ratings for TV Shows:", df[df['type'] == 'TV Show']['rating'].value_counts().head(5).to_dict())

plt.figure(figsize=(13, 5))
sns.countplot(data=df, x='rating', hue='type', order=rating_type.index, palette=['#E50914', '#221F1F'])
plt.title('Ratings for Movies vs TV Shows')
plt.xlabel('Rating')
plt.ylabel('Number of Titles')
plt.xticks(rotation=45)
plt.legend(title='Type')
plt.show()
```

type	Movie	TV Show
rating		
TV-MA	2062	1145
TV-14	1427	733
TV-PG	540	323
R	797	2
PG-13	490	0
TV-Y7	139	195
TV-Y	131	176
PG	287	0
TV-G	126	94
NR	75	5
G	41	0
TV-Y7-FV	5	1
NC-17	3	0
UR	3	0

```
Top 5 ratings for Movies  : {'TV-MA': 2062, 'TV-14': 1427, 'R': 797, 'TV-PG': 540, 'PG-13': 490}
Top 5 ratings for TV Shows: {'TV-MA': 1145, 'TV-14': 733, 'TV-PG': 323, 'TV-Y7': 195, 'TV-Y': 176}
```



Task 11: What are the oldest and newest titles available on Netflix?

```
cols = ['title', 'type', 'release_year', 'date_added', 'country']
print("10 oldest titles (by release year):")
display(df.sort_values('release_year').head(10)[cols])

print("10 newest titles (by release year):")
display(df.sort_values(['release_year', 'date_added'], ascending=False).head(10)[cols])

oldest = df.loc[df['release_year'].idxmin()]
newest = df.loc[df['date_added'].idxmax()]
print(f"Oldest title : {oldest['title']} ({oldest['type']}, released {oldest['release_year']})")
print(f"Most recently added title: {newest['title']} (added {newest['date_added'].date()}, released {newest['release_year']})")
```

```
10 oldest titles (by release year):
```

	title	type	release_year	date_added	country
4250	Pioneers: First Women Filmmakers*	TV Show	1925	2018-12-30	NaN
7790	Prelude to War	Movie	1942	2017-03-31	United States
8205	The Battle of Midway	Movie	1942	2017-03-31	United States
8763	WWII: Report from the Aleutians	Movie	1943	2017-03-31	United States
8739	Why We Fight: The Battle of Russia	Movie	1943	2017-03-31	United States
8660	Undercover: How to Operate Behind Enemy Lines	Movie	1943	2017-03-31	United States
8436	The Negro Soldier	Movie	1944	2017-03-31	United States
8419	The Memphis Belle: A Story of a\nFlying Fortress	Movie	1944	2017-03-31	United States
8640	Tunisian Victory	Movie	1944	2017-03-31	United States, United Kingdom
7575	Nazi Concentration Camps	Movie	1945	2017-03-31	United States

```
10 newest titles (by release year):
```

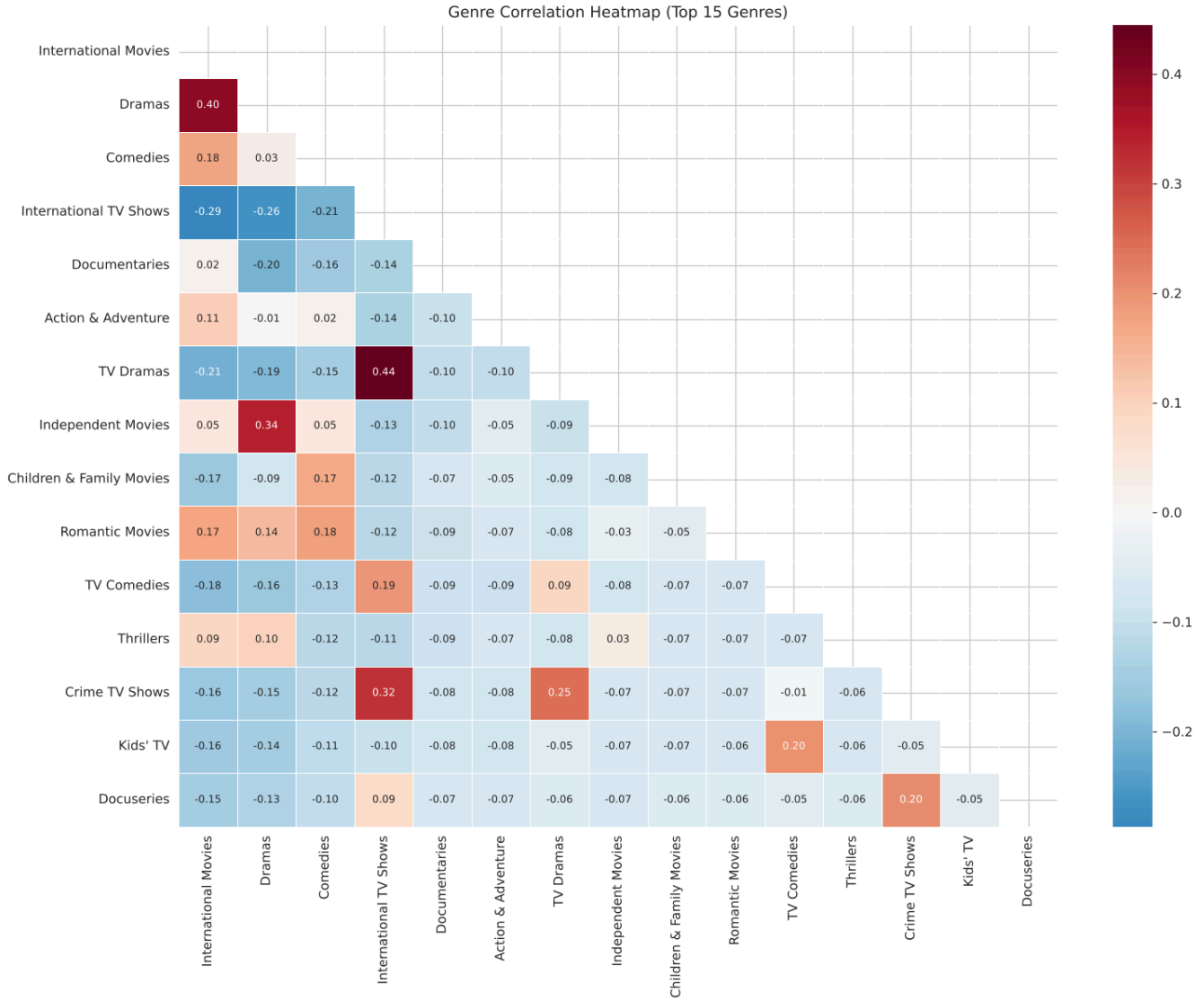
	title	type	release_year	date_added	country
1	Blood & Water	TV Show	2021	2021-09-24	South Africa
2	Ganglands	TV Show	2021	2021-09-24	NaN
3	Jailbirds New Orleans	TV Show	2021	2021-09-24	NaN
4	Kota Factory	TV Show	2021	2021-09-24	India
5	Midnight Mass	TV Show	2021	2021-09-24	NaN
6	My Little Pony: A New Generation	Movie	2021	2021-09-24	NaN
8	The Great British Baking Show	TV Show	2021	2021-09-24	United Kingdom
9	The Starling	Movie	2021	2021-09-24	United States
10	Vendetta: Truth, Lies and The Mafia	TV Show	2021	2021-09-24	NaN
11	Bangkok Breaking	TV Show	2021	2021-09-23	NaN

```
Oldest title : Pioneers: First Women Filmmakers* (TV Show, released 1925)
Most recently added title: Dick Johnson Is Dead (added 2021-09-25, released 2020)
```

Task 12: Which genres frequently co-occur together? (Genre correlation heatmap)

```
genre_dummies = df['listed_in'].str.get_dummies(sep=', ')
top15_genres = genre_dummies.sum().sort_values(ascending=False).head(15).index
genre_corr = genre_dummies[top15_genres].corr()

plt.figure(figsize=(14, 11))
mask = np.triu(np.ones_like(genre_corr, dtype=bool))
sns.heatmap(genre_corr, mask=mask, annot=True, fmt='.2f', cmap='RdBu_r', center=0,
            linewidths=0.5, annot_kws={'size': 8})
plt.title('Genre Correlation Heatmap (Top 15 Genres)')
plt.tight_layout()
plt.show()
```




```
co_occurrence = genre_dummies[top15_genres].T.dot(genre_dummies[top15_genres])
pairs = []
for i, g1 in enumerate(top15_genres):
    for g2 in top15_genres[i + 1:]:
        pairs.append([g1, g2, co_occurrence.loc[g1, g2]])
pairs_df = (pd.DataFrame(pairs, columns=['Genre 1', 'Genre 2', 'Titles Together'])
            .sort_values('Titles Together', ascending=False).head(10).reset_index(drop=True))
print("Top 10 genre pairs that occur together most often:")
display(pairs_df)
```

Top 10 genre pairs that occur together most often:

	Genre 1	Genre 2	Titles Together
0	International Movies	Dramas	1483
1	International Movies	Comedies	804
2	Dramas	Independent Movies	588
3	International TV Shows	TV Dramas	514
4	Dramas	Comedies	502

	Genre 1	Genre 2	Titles Together
5	International Movies	Action & Adventure	398
6	International Movies	Romantic Movies	372
7	Dramas	Romantic Movies	308
8	International TV Shows	Crime TV Shows	299
9	International Movies	Documentaries	296